

EPIC 06 — Task 02: Transactions

The task asks you to study and implement:

- Local transactions
- Distributed transactions
- Transaction boundaries
- Proper transaction handling

We will continue with the **Order Service, Product Service, and Notification Service** from the previous tasks.

Step 1 — Local Transaction

A **local transaction** happens inside one service and one database.

For example, when creating an order:

Order Service

↓

Order Database

↓

BEGIN TRANSACTION

↓

Save Order

↓

Save Order Items

↓

COMMIT

If something fails:

BEGIN

↓

Save Order

↓

Save Order Item ❌

↓

ROLLBACK

Step 2 — Add @Transactional

In your **Order Service**, open:

OrderService.java

Use:

```
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
```

```
@Service
```

```
public class OrderService {
```

```
    private final OrderRepository orderRepository;
```

```
    private final OrderItemRepository orderItemRepository;
```

```
    public OrderService(
        OrderRepository orderRepository,
        OrderItemRepository orderItemRepository) {
```

```
        this.orderRepository = orderRepository;
```

```
        this.orderItemRepository = orderItemRepository;
```

```
    }
```

```
@Transactional
```

```

public Order createOrder(Order order) {

    Order savedOrder = orderRepository.save(order);

    // Save order items here
    // If an error occurs, the transaction is rolled back.

    return savedOrder;
}
}

```

The important annotation is:

`@Transactional`

It tells Spring to treat the method as **one transaction**.

Step 3 — Transaction Boundary

The transaction boundary should normally be at the **service layer**.

Controller

↓

OrderService

|

| @Transactional

|

|— Save Order

|

|— Save Order Items

|

└─ Commit / Rollback

Don't put the transaction boundary in the Controller

Prefer:

@Transactional

```
public void createOrder() {  
  
    // database operations  
  
}
```

instead of:

```
@Transactional  
@GetMapping("/orders")  
public ...
```

The service layer should control the business transaction.

Step 4 — Rollback Example

Suppose the application performs:

1. Create Order 
2. Create Order Item 
3. Update Order 

Because all operations are inside:

@Transactional

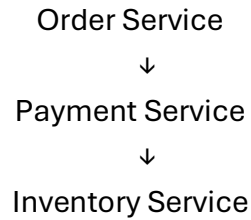
the database performs:

```
Order created    → ROLLBACK  
Order Item created → ROLLBACK  
Order update     → ROLLBACK
```

The database returns to its previous consistent state.

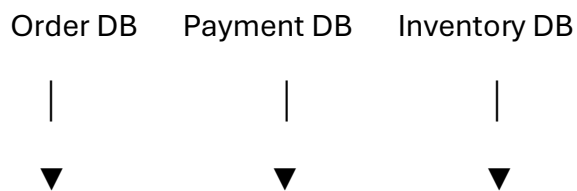
Step 5 — Distributed Transaction

In microservices, a transaction may involve multiple services:



Each service has its **own database**.

For example:



Order Service → Payment Service → Inventory Service

A single normal @Transactional transaction **cannot automatically span all three independent service databases**.

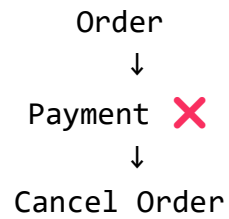
For this type of workflow, a distributed transaction pattern such as the **Saga pattern** is commonly used.

Step 6 — Example Distributed Workflow

Consider:



If payment fails:



If inventory fails after payment succeeds:

Order

Payment

Inventory

↓

Refund Payment

↓

Cancel Order

These are **compensating actions** rather than one database rollback.

Step 7 — Order Service Transaction

Your Order Service can use:

@Transactional

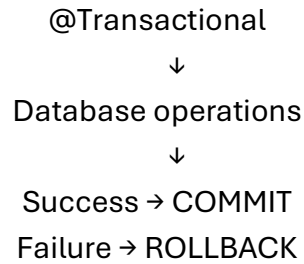
```
public Order createOrder(Order order) {
```

```
    Order savedOrder = orderRepository.save(order);
```

```
    return savedOrder;
```

```
}
```

The local database transaction finishes when the method successfully completes.



Step 8 — Important Transaction Rule

Keep the transaction boundary around **database operations that belong together**.

For example:

```
@Transactional
public void createOrderWithItems(
    Order order,
    List<OrderItem> items) {

    Order savedOrder = orderRepository.save(order);

    for (OrderItem item : items) {
        item.setOrderId(savedOrder.getId());
        orderItemRepository.save(item);
    }
}
```

If one order item fails, the local transaction rolls back the order and previously saved items.

Step 9 — Don't Keep Long Network Calls Inside a DB Transaction

Avoid doing this:

```
@Transactional
```

```
public void createOrder() {
```

```
    orderRepository.save(order);
```

```
    // REST call
```

```
    paymentService.processPayment();
```

```
    // Kafka call
```

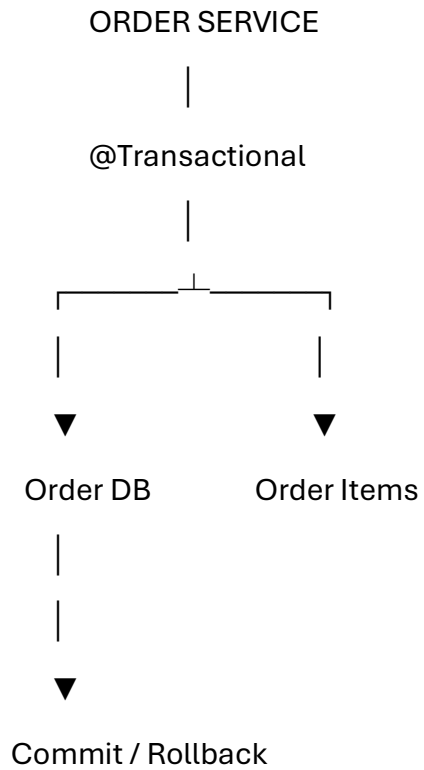
```
    kafkaTemplate.send(...);
```

```
}
```

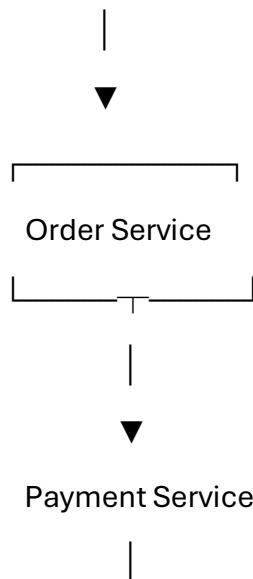
The database transaction remains open while waiting for external services.

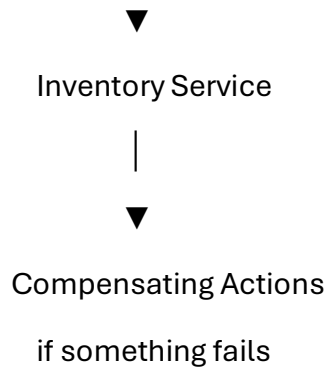
A better design is to keep the local DB transaction focused and use events/Saga-style coordination for cross-service operations.

Final Architecture for Task 02



Distributed Workflow





Requirement	Implementation
Local transactions	@Transactional
Distributed transactions	Saga/compensating actions
Transaction boundaries	Service layer
Rollback	Automatic on transaction failure
Multiple microservices	Independent DB transactions
Proper handling	Short local transactions + Saga for cross-service workflow

Task 02 — Transactions are completed.

